Mongo Database Assignment

SUBMITTED BY

Hifza Alvi

ROLL NO: 1250259026

Master's of Computer Application (MCA DS&AI 1)

BABU BANARASI DAS UNIVERSITY

Lucknow, Uttar Pradesh

UNDER THE GUIDANCE OF

ANKIT SONI

(FACULTY , BBDU)

------Importing Library-------

```
#libraries
from pymongo import MongoClient
from dotenv import load_dotenv
import os
```

--Loading Environment Variables & Connecting to MongoDB Atlas--

```
# Load .env
load_dotenv(".env")

MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")

# Connection to mongodb atlas
client = MongoClient(MONGO_URI)
# it will ensure connection is working or not
client.admin.command("ping")

db = client[DB_NAME]

print("Connected to MongoDB Atlas")
print("DB:", DB_NAME)
```

Output:- Connected to MongoDB Atlas

DB: CollegeDB

-------Uploading data to atlas -------

```python
#upload data to atlas
import json

files = {
    "students": "students_full.json",
    "faculty": "faculty_full.json",
    "courses": "courses_full.json",
    "enrollments": "enrollments_full.json",
    "activities": "activities_full.js
    on"
}

for col, file in files.items():
    try:
        with open(file, "r") as f:
            data = json.load(f)

        db[col].delete_many({})
        db[col].insert_many(data)

        print(f"{col}: {len(data)} inserted")

    except Exception as e:
        print(f"{col} error:", e)
```

Output:-

```
students: 100 inserted
faculty: 100 inserted
courses: 100 inserted
enrollments: 100 inserted
activities: 100 inserted
```

------Verifying data is loaded or not -------

```
# verifying data is loaded or not
for col in files:
    print(col, db[col].count_documents({}))
```

Output:-

```
students 100
faculty 100
courses 100
enrollments 100
activities 100
```

1. Complex Filters & Projections

Q1. List the names and departments of students who have more than 85% attendance and are skilled in both "MongoDB" and "Python".

QUERY-

```python
# Name: Hifza Alvi
# Roll no-1250259026
import json

# Query conditions
query = {
    "attendance": {"$gt": 85},
    "skills": {"$all": ["MongoDB", "Python"]}
}

# fields to show
projection = {
    "_id": 0,
    "name": 1,
    "department": 1
}

# Fetch data from MongoDB
students = list(db.students.find(query, projection))

# Total count
print("Total Students:", len(students))

# Print formatted output
for student in students:
    print(json.dumps(student, indent=4))
```

Output: Total Students: 0

Q2. Show all faculty who are teaching more than 2 courses. Display their names and the total number of courses they teach.

QUERY-

```
import json

# Aggregation Pipeline
pipeline = [
    {
        "$project": {
            "_id": 0,
            "name": 1,
            "total_courses": {"$size": "$courses"}
        }
    },
    {
        "$match": {
            "total_courses": {"$gt": 2}
        }
    }
]

#  Execution of aggregation
result = list(db.faculty.aggregate(pipeline))

# Count
print(" Total Faculty:", len(result))

# Print formatted output using indent for better json view
for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
Total Faculty: 9
{
    "name": "Charles Newton",
    "total_courses": 3
}
{
    "name": "Julia Cole",
    "total_courses": 3
}
{
    "name": "Darrell Velasquez",
    "total_courses": 3
}
{
    "name": "Michael Poole",
    "total_courses": 3
}
{
    "name": "John Duran",
    "total_courses": 3
}
{
    "name": "Daniel Allen",
    "total_courses": 3
}
...
{
    "name": "Robert Lara",
    "total_courses": 3
}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

2. Joins ($lookup) and Aggregations

Q3. Write a query to show each student's name along with the course titles they are enrolled in (use $lookup between enrolments, students, and courses).

QUERY-

```
pipeline = [
    {
        "$lookup": {
            "from": "students",
            "localField": "student_id",
            "foreignField": "_id",
            "as": "student"
        }
    },
    {"$unwind": "$student"},
    {
        "$lookup": {
            "from": "courses",
            "localField": "course_id",
            "foreignField": "_id",
            "as": "course"
        }
    },
    {"$unwind": "$course"},
    {
        "$group": {
            "_id": "$student.name",
            "course_titles": {"$addToSet": "$course.title"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "student_name": "$_id",
            "course_titles": 1
        }
    }
]

result = list(db.enrollments.aggregate(pipeline))

print("Total Students:", len(result))

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
Total Students: 66
{
    "course_titles": [
        "Advanced analyzing budgetary management",
        "Streamlined bandwidth-monitored structure",
        "Persevering asynchronous hub",
        "Customer-focused cohesive info-mediaries",
        "Digitized disintermediate orchestration"
    ],
    "student_name": "Ronald Trevino"
}
{
    "course_titles": [
        "Optional neutral workforce"
    ],
    "student_name": "Gabriela Le"
}
{
    "course_titles": [
        "De-engineered well-modulated installation"
    ],
    "student_name": "Daniel Brown"
}
{
    "course_titles": [
...
        "Triple-buffered cohesive frame"
    ],
    "student_name": "Brian Russell"
}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Q4. For each course, display the course title, number of students enrolled, and average marks (use $group).

QUERY-

```
pipeline = [
    {
        "$group": {
            "_id": "$course_id",
            "students_enrolled": {"$sum": 1},
            "average_marks": {"$avg": "$marks"}
        }
    },
    {
        "$lookup": {
            "from": "courses",
            "localField": "_id",
            "foreignField": "_id",
            "as": "course"
        }
    },
    {"$unwind": "$course"},
    {
        "$project": {
            "_id": 0,
            "course_title": "$course.title",
            "students_enrolled": 1,
            "average_marks": 1
        }
    }
]

result = list(db.enrollments.aggregate(pipeline))

print("Total Courses:", len(result))

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
Total Courses: 66
{
    "students_enrolled": 1,
    "average_marks": 97.0,
    "course_title": "Triple-buffered eco-centric implementation"
}
{
    "students_enrolled": 1,
    "average_marks": 92.0,
    "course_title": "Quality-focused local leverage"
}
{
    "students_enrolled": 1,
    "average_marks": 75.0,
    "course_title": "De-engineered well-modulated installation"
}
{
    "students_enrolled": 1,
    "average_marks": 67.0,
    "course_title": "Focused multi-state encoding"
}
{
    "students_enrolled": 1,
    "average_marks": 90.0,
    "course_title": "Cloned intermediate ability"
...
    "students_enrolled": 2,
    "average_marks": 64.0,
    "course_title": "Persistent static migration"
}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

3. Grouping, Sorting, and Limiting

Q5. Find the top 3 students with the highest average marks across all enrolled courses.

QUERY-

```
pipeline = [
    {
        "$group": {
            "_id": "$student_id",
            "avg_marks": {"$avg": "$marks"}
        }
    },
    {
        "$lookup": {
            "from": "students",
            "localField": "_id",
            "foreignField": "_id",
            "as": "student"
        }
    },
    {"$unwind": "$student"},
    {
        "$project": {
            "_id": 0,
            "student_name": "$student.name",
            "avg_marks": 1
        }
    },
    {"$sort": {"avg_marks": -1}},
    {"$limit": 3}
]

result = list(db.enrollments.aggregate(pipeline))

print(" Top 3 Students:\n")

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
 Top 3 Students:

{
    "avg_marks": 100.0,
    "student_name": "Diane Phillips"
}
{
    "avg_marks": 98.0,
    "student_name": "Brandon Rios"
}
{
    "avg_marks": 94.0,
    "student_name": "Larry Ramsey"
}
```

Q6. Count how many students are in each department. Display the department with the highest number of students.

QUERY-

```
pipeline = [
    {
        "$group": {
            "_id": "$department",
            "count": {"$sum": 1}
        }
    },
    {
        "$sort": {
            "count": -1
        }
    }
]

result = list(db.students.aggregate(pipeline))

print(" All Departments:\n")

for doc in result:
    output = {
        "department": doc["_id"],
        "student_count": doc["count"]
    }
    print(json.dumps(output, indent=4))

#  Top department
if result:
    top = {
        "department": result[0]["_id"],
        "student_count": result[0]["count"]
    }

    print("\n Department with Highest Students:\n")
    print(json.dumps(top, indent=4))
```

Output:

```
 All Departments:

{
    "department": "Electrical",
    "student_count": 23
}
{
    "department": "Civil",
    "student_count": 22
}
{
    "department": "Computer Science",
    "student_count": 20
}
{
    "department": "Biotechnology",
    "student_count": 19
}
{
    "department": "Mechanical",
    "student_count": 16
}

 Department with Highest Students:

{
    "department": "Electrical",
    "student_count": 23
}
```

4. Update, Upsert, and Delete

Q7. Update attendance to 100% for all students who won any "Hackathon".

QUERY-

```
# Name: Hifza Alvi
# Roll No: 1250259026


# Finding all student IDs who won Hachathon
winner_ids = db.activities.distinct(
    "student_id",
    {
        "type": "Hackathon",
        "position":"Winner"
    }
)

# Update attendance to 100%
update_result = db.students.update_many(
    {
        "_id": {"$in":winner_ids}
    },
        "$set": {"attendance":100}
)   }

# result
output = {
    "matched_students": update_result.matched_count,
    "updated_students": update_result.modified_count
}

print("\n Result:")
print(json.dumps(output, indent=4))
```

Output:

```
 Result:
{
    "matched_students": 9,
    "updated_students": 0
}
```

Q8. Delete all student activity records where the activity year is before 2022.

QUERY-

```python
# Name: Hifza Alvi
# Roll No: 1250259026

# Delete records where year < 2022
delete_result = db.activities.delete_many(
    {
        "year": {"$lt": 2022}
    }
)

output = {
    "deleted_records": delete_result.deleted_count
}

print("\n Result:")
print(json.dumps(output,indent=4))
```

Output:

```
Result:
{
    "deleted_records": 0
}
```

Q9. Upsert a course record for "Data Structures" with ID "C150" and credits 4—if it doesn't exist, insert it; otherwise update its title to "Advanced Data Structures".

```python
# Name: Hifza Alvi
# Roll No: 1250259026

import json

result = db.courses.update_one(
    {"_id": "C150"},
        {
            "$set": {
                "title": "Advanced Data Structures"
            },
            "$setOnInsert": {
                "credits": 4
        },
    },
    upsert=True
)

course = db.courses.find_one({"_id": "C150"}, {"_id: 0)

print("\nResult:")
print(json.dumps(course, indent=4))
```

QUERY-

Output:

```
Result:
{
    "credits": 4,
    "title": "Advanced Data Structures"
}
```

5. Array & Operator Usage

Q10. Find all students who have "Python" as a skill but not "C++".

QUERY-

```python
# Name: Hifza Alvi
# Roll No: 1250259026

import json

query = {
    "skills": "Python",
    "skills": {"$nin": ['C++']}
}

projection = {
    "_id": 0,
    "name": 1,
    "skills": 1
},

result = list(db.students.find(query, projection))

print("\nQ10 Result:")
print("Total:", len(result))

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
Q10 Result:
Total: 77
{
    "name": "Bruce Blair",
    "skills": [
        "MongoDB",
        "Linux"
    ]
}
{
    "name": "Alexandra Bailey",
    "skills": [
        "Research",
        "AutoCAD"
    ]
}
{
    "name": "Kyle Hale",
    "skills": [
        "Python",
        "Java"
    ]
}
{
...
        "MongoDB",
        "JavaScript"
    ]
}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Q11. Return names of students who participated in "Seminar" and "Hackathon" both.

QUERY-

```
import json

pipeline = [
    {
        "$group": {
            "_id": "$student_id",
            "types": {"$addToSet": "$type"}
        }
    },
    {
        "$match": {
            "types": {"$all": ["Seminar", "Hackathon"]}
        }
    },
    {
        "$lookup": {
            "from": "students",
            "localField": "_id",
            "foreignField": "_id",
            "as": "student"
        }
    },
    {"$unwind": "$student"},
    {
        "$project": {
            "_id": 0,
            "name": "$student.name"
        }
    }
]

result = list(db.activities.aggregate(pipeline))

print("\n Result:")

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
 Result:
{
    "name": "Adam Solomon"
}
{
    "name": "Patricia Scott"
}
{
    "name": "Taylor Webb"
}
{
    "name": "Lydia Day"
}
{
    "name": "Carlos Bryant"
}
```

6. Subdocuments and Nested Conditions

Q12. Find students who scored more than 80 in "Web Development" only if they belong to the "Computer Science" department.

QUERY-

```
import json

pipeline = [
    {
        "$group": {
            "_id": "$student_id",
            "types": {"$addToSet": "$type"}
        }
    },
    {
        "$match": {
            "types": {"$all": ["Seminar", "Hackathon"]}
        }
    },
    {
        "$lookup": {
            "from": "students",
            "localField": "_id",
            "foreignField": "_id",
            "as": "student"
        }
    },
    {"$unwind": "$student"},
    {
        "$project": {
            "_id": 0,
            "name": "$student.name"
        }
    }
]

result = list(db.activities.aggregate(pipeline))

print("\n Result:")

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output: Result : Total:0

7. Advanced Aggregation (Challenge Level)

Q13. For each faculty member, list the names of all students enrolled in their courses along with average marks per student per faculty.

QUERY-

```
# Name: Hifza Alvi
# Roll No: 1250259026

import json

pipeline = [
    {
        "$lookup": {
            "from": "courses",
            "localField": "_id",
            "foreignField": "faculty_id",
            "as": "courses"
        }
    },
    {"$unwind": "$courses"},
    {
        "$lookup": {
            "from": "enrollments",
            "localField": "courses._id",
            "foreignField": "course_id",
            "as": "enrollments"
        }
    },
    {"$unwind": "$enrollments"},
    {
        "$lookup": {
            "from": "students",
            "localField": "enrollments.student_id",
            "foreignField": "_id",
            "as": "student"
        }
    },
    {"$unwind": "$student"},
    {
        "$group": {
            "_id": {
                "faculty": "$name",
                "student": "$student.name"
            },
            "avg_marks": {"$avg": "$enrollments.marks"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "faculty_name": "$_id.faculty",
            "student_name": "$_id.student",
            "avg_marks": 1
        }
    },
    {
        "$sort": {
            "faculty_name": 1,
            "student_name": 1
        }
    }
]

result = list(db.faculty.aggregate(pipeline))

print("\n Result:")
print("Total rows:", len(result))

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
 Result:
Total rows: 99
{
    "avg_marks": 90.0,
    "faculty_name": "Alexis Stone",
    "student_name": "Anthony Zavala"
}
{
    "avg_marks": 93.0,
    "faculty_name": "Alexis Stone",
    "student_name": "Barbara Jones"
}
{
    "avg_marks": 69.0,
    "faculty_name": "Andrew Mcmahon",
    "student_name": "Dr. Michael Griffin Jr."
}
{
    "avg_marks": 81.0,
    "faculty_name": "Andrew Mcmahon",
    "student_name": "Megan Taylor"
}
{
    "avg_marks": 52.0,
...
    "avg_marks": 90.0,
    "faculty_name": "William Alexander",
    "student_name": "Vincent Norris"
}
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Q14. Show the most popular activity type (e.g., Hackathon, Seminar, etc.) by number of student participants.

QUERY-

```
# Name: Hifza Alvi
# Roll No: 1250259026

import json

pipeline = [
    {
        "$group": {
            "_id": "$type",
            "students": {"$addToSet": "$student_id"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "activity_type": "$_id",
            "participants": {"$size": "$students"}
        }
    },
    {
        "$sort": {
            "participants": -1
        }
    },
    {
        "$limit": 1
    }
]

result = list(db.activities.aggregate(pipeline))

print("\n Result:")

for doc in result:
    print(json.dumps(doc, indent=4))
```

Output:

```
 Result:
{
    "activity_type": "Hackathon",
    "participants": 29
}
```